

A model-fitting and validation workflow for growth, drug response, coculture, and multi-population ODE models.

- Fits noisy growth data with error-aware joint objectives and bounded optimization.
- Compares biological model hypotheses using BIC and diagnostic outputs.
- Carries parameters from untreated monoculture into treated and coculture stages.
- Adds practical and symbolic identifiability analysis before interpreting real-data parameters.



DATA CONTRACT

One long-format table supports the standard workflows

Use a single DataFrame for every observed series. Error bars require replicate-level data or a defensible uncertainty column.

Column	Type	Role
time	Float64	observation time
count	Float64	observed population or burden
error	Float64	measurement SD for weighting and error bars
dose	Float64	known treatment amount
culture_type	String	monoculture or coculture
population_type	String	sensitive, resistant, damaged, macrophage
replicate	Int	biological or assay replicate stratum

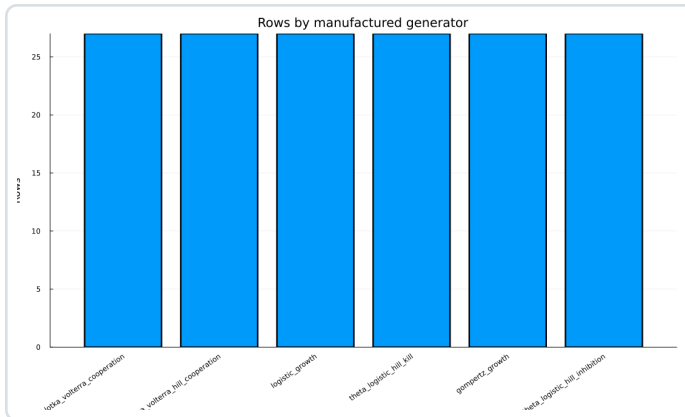
Keep individual replicates when possible. The package can summarize uncertainty, but it cannot invent independent biological information from a single trajectory.

```
df = DataFrame(time, count, error, dose, culture_type,  
               population_type, replicate, cell_line, density)
```

MODEL LIBRARY

Start simple, then add biology only when data support it

Every reported graphic should name the fitted function and which process it represents.



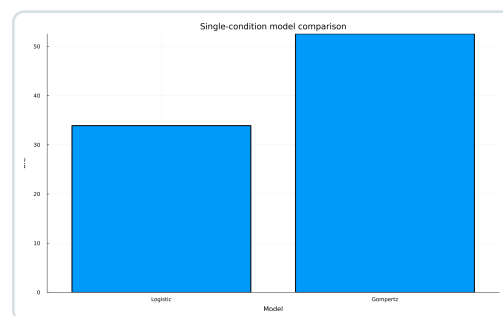
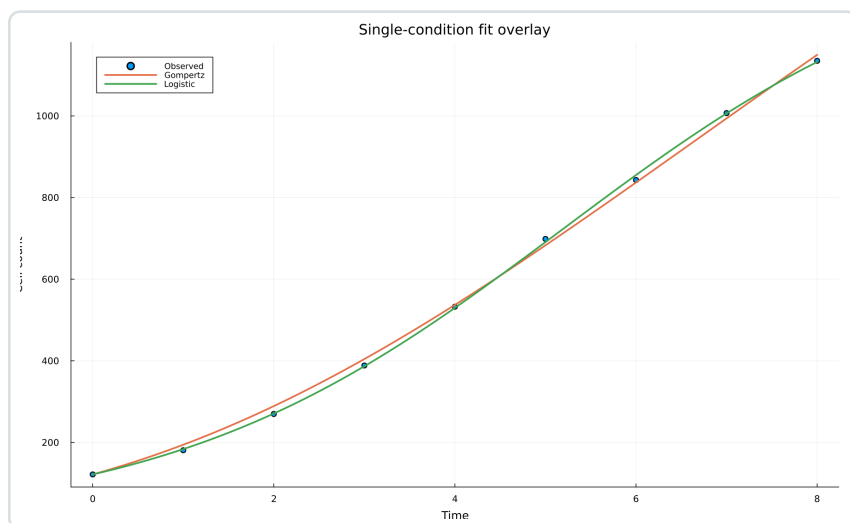
EXAMPLE MODEL LADDER

- Logistic: $dN/dt = r N (1 - N/K)$.
- Gompertz and exponential: alternative single-population growth hypotheses.
- Drug effect: $dN/dt = \text{growth} - \text{kill}(\text{dose, parameters}) N$.
- Competition: each population's growth is reduced by the other population.
- Compartmental models: viable, damaged, terminally damaged, and immune-cell states.

UNTREATED MONOCULTURE

Manufactured sensitive and resistant data are fit against growth models

The study story begins with noisy logistic-like trajectories. Candidate models are logistic, Gompertz, exponential, and generalized logistic growth.

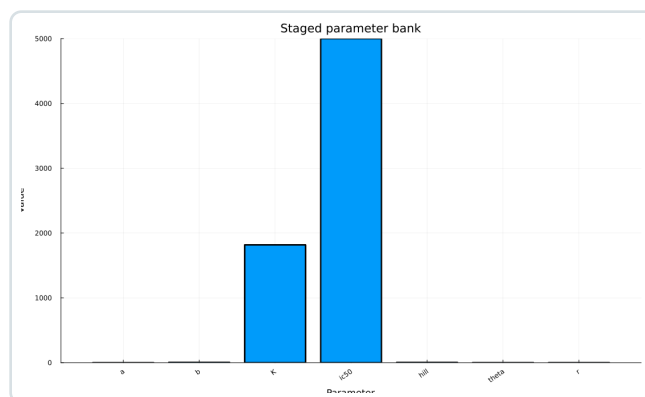
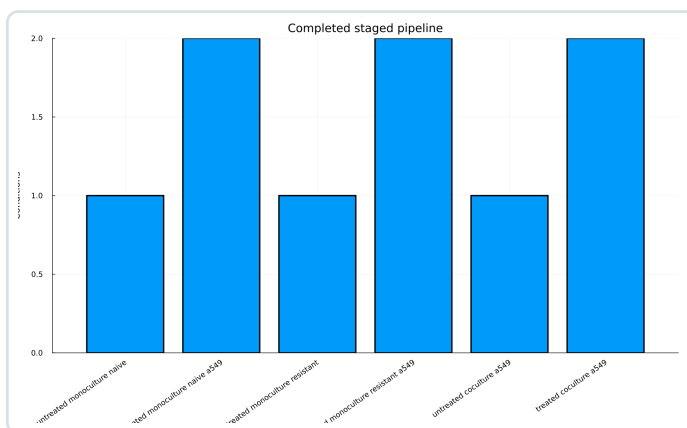


Functions used: `compare_models` / `compare_models_dict`, followed by BIC ranking. The selected model and fitted parameters are displayed with the overlay.

STAGE-WISE BIOLOGY

Treated and coculture fits inherit what was learned upstream

The manufactured data change one biological layer at a time, so later fits can anchor or initialize shared growth parameters from earlier conditions.



- Treated monoculture adds a drug-response term.
- Untreated coculture adds competition effects.
- Treated coculture combines growth, drug, and competition terms.
- Each stage repeats candidate-model comparison and BIC selection.

FITTING CHOICES

Choose the API that matches the scientific question

Use the compact fitting helpers for focused questions and joint fitting when multiple series must constrain the same parameter vector.

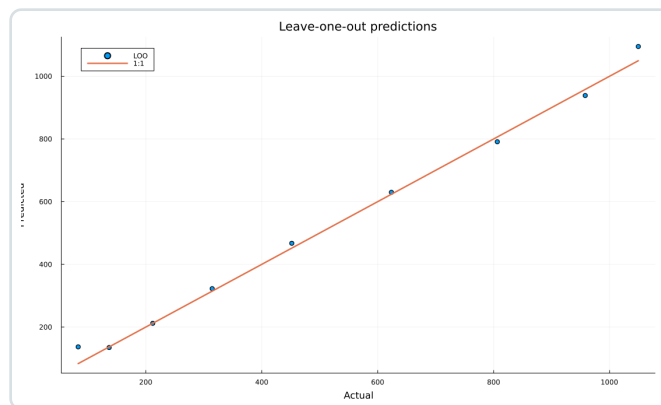
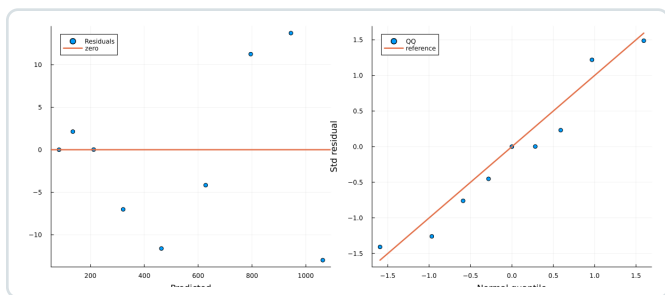
Question	Primary function	Why it helps
One registered model	<code>fit_model</code>	bounded fit to one trajectory
One condition from a table	<code>fit_condition</code>	preserves condition metadata
Two hypotheses	<code>compare_models</code>	direct BIC decision
Two experiments	<code>compare_datasets</code>	separate fit summaries
Shared multi-state model	<code>run_joint_fit</code>	one likelihood across series
Difficult objective	<code>run_joint_multistart</code>	audits multiple initial guesses

The companion Fitting Varieties PDF gives function signatures, typed arguments, and example use cases for these APIs.

ERROR BARS AND VALIDATION

Plots should carry experimental uncertainty, not cosmetic confidence

Provide an error column derived from technical or biological replicates. Residual diagnostics and held-out predictions then test whether the model earns its apparent fit.

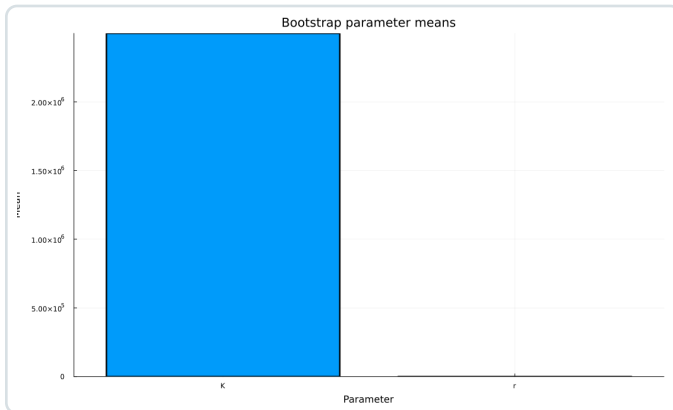


- Residual analysis checks systematic over- or under-prediction.
- Leave-one-out and k-fold tests evaluate predictive stability.
- BIC chooses among candidate models; it does not establish parameter uniqueness.

BOOTSTRAP UNCERTAINTY

Refit many plausible versions of the experiment

The package includes stage-level bootstrap summaries and a new joint-model bootstrap for identifiability analysis.



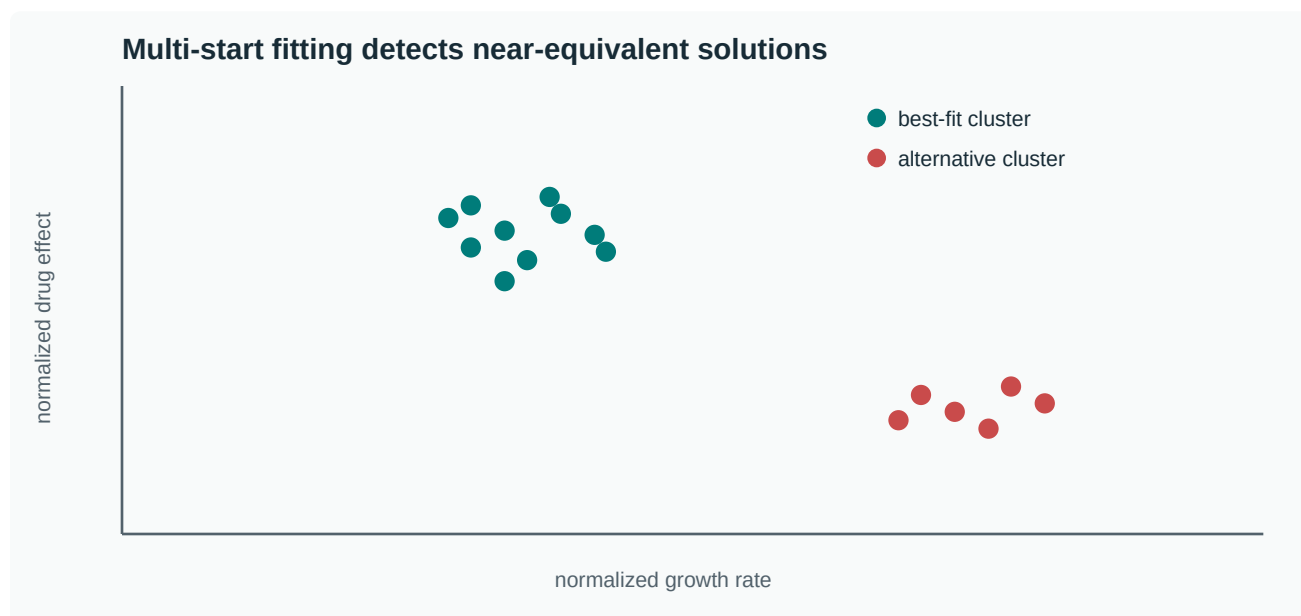
NEW JOINT BOOTSTRAP

- `bootstrap_joint_fit(...; method=:residual)` resamples residuals within each observed series.
- `method=:parametric` draws noise using each series `residual_scale`.
- The output records failed refits, success rate, parameter means, SDs, and 95% intervals.
- Separate replicate strata in `dataset_specs` so the resampling unit remains scientifically meaningful.

PRACTICAL IDENTIFIABILITY

A low error does not mean that parameters are uniquely determined

The new practical-identifiability report combines fit stability, local information, profile likelihoods, and bootstrap evidence.

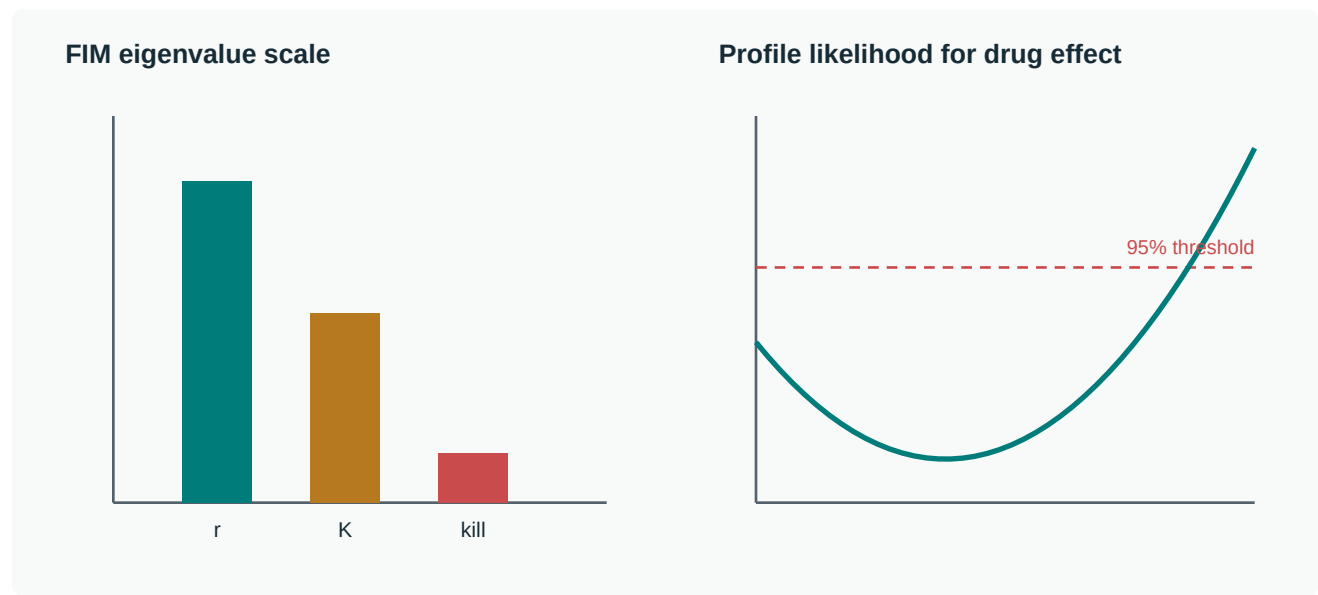


- `generate_multistarts` creates broad, reproducible starting points within scientific bounds.
- Near-equivalent but separated solution clusters are a warning that data do not distinguish parameter combinations.
- `practical_identifiability_report` returns `passes_numerical_gates` only when conservative numerical conditions agree.

FIM AND PROFILES

Diagnose local confounding and whether data bracket each parameter

The Fisher information matrix is local. Profile likelihood refits all other parameters while one parameter is held fixed.



Function	Output	Interpretation
fisher_information	rank and condition number	rank loss or huge conditioning signals local confounding
profile_likelihood	profile + confidence status	unbounded profiles need more data or a simpler model
prediction_jacobian	sensitivity matrix	documents how each observation responds to parameters

STRUCTURAL IDENTIFIABILITY

Prove what the model can identify only after defining what the assay observes

Symbolic global/local analysis is deliberately separate from numerical fitting because the observation process changes the answer.

REQUIRED OBSERVATION MAP

```
map = ObservationMap(  
  "drug_macrophage_four_state",  
  [:S, :D1, :D2, :M],  
  [:viable_cells, :recoverable_damage, :terminal_damage, :macrophages],  
)
```

- `validate_observation_map` checks every joint-fit series against the documented measured states or observables.
- `structural_identifiability` calls `StructuralIdentifiability.jl` for symbolic global or local classification.
- `structural_identifiability_report` never relabels a numerical fit as a structural proof.

FOUR-POPULATION EXAMPLE

Viable, damaged, terminal-damage, and macrophage compartments

This is a supported joint-fitting and identifiability pattern, not a claim that every seven-parameter model will be estimable from every experiment.

```
function four_state_model!(du, u, p, t)
    S, D1, D2, M = u
    growth, K, drug_damage, repair, terminal, clearance, kill = p
    du[1] = growth*S*(1-(S+D1)/K) - drug_damage*drug(t)*S - kill*M*S
    du[2] = drug_damage*drug(t)*S - repair*D1 - terminal*D1
    du[3] = terminal*D1 - clearance*M*D2
    du[4] = 0.0
end
```

- Measure each compartment directly when possible; total viability alone can confound damage, repair, and clearance.
- Use `residual_scale` for each assay and test recovery at the real sampling schedule before interpreting a biological mechanism.
- Known drug concentration belongs in the symbolic model as a known input, not an estimated hidden state.

REAL-DATA READINESS

The package now supports a defensible analysis workflow

Release v0.5.0 adds the identifiability layer needed before applying complex models to sparse, noisy experiments.

- Fit a scientifically constrained model with multiple starts and realistic solver tolerances.
- Use BIC, residual checks, held-out prediction, and bootstrap stability for model selection and uncertainty.
- Gate interpretation on FIM rank/conditioning, solution clusters, profile bounds, and bootstrap success.
- Run synthetic recovery benchmarks before committing to a real experimental design.
- For a structural claim, repeat the exact ODE outputs and known inputs in `StructuralIdentifiability.jl`.

Release status: v0.5.0 has been pushed and submitted to Julia General Registry for automated validation. The local Windows environment has an Application Control restriction that blocks Julia precompiled package DLLs, so full local runtime tests remain environment-blocked.